Registration Link: https://forms.gle/EEy5g41B18kLE7xu6

Event: Hack Orbit 2026

Problem Statements

The seven problem statements span four tracks. Teams select **one problem statement** to work on.

Track 1 — AI & Human Augmentation

PS-01 | SkillForge AI: Personalized Learning & Career Mentor

Difficulty: Medium | Domain: AI + EdTech

Background:
Students often struggle to identify skill gaps, prepare personalized learning roadmaps, and build industry-ready portfolios. Existing platforms provide generic recommendations instead of adaptive guidance.

Problem:
Build an AI-powered career and learning platform that analyzes a student's skills, interests, resume, and project portfolio to generate personalized learning paths, recommend relevant courses and projects, suggest interview preparation resources, and track progress through interactive dashboards.

Constraints:

- • Support resume upload and skill extraction.
- • Generate personalized learning roadmaps.
- • Recommend projects, certifications, and interview questions.
- • Include progress tracking and analytics.

Expected Output:
A web platform that helps students plan, learn, and prepare for placements using AI-driven recommendations.

Judging Emphasis: Recommendation quality, usability, AI integration, personalization, and overall user experience.

PS-02 | FakeShield: Real-Time Deepfake Detection for Live Video Calls

Difficulty: Medium-High | Domain: AI + Cybersecurity + Computer Vision

Background:
Deepfake attacks during video calls (identity fraud in KYC, financial meetings, academic exams) are a growing threat. Existing detectors operate offline on recorded videos, leaving live calls completely unprotected.

Problem:
Build a real-time deepfake detection system that integrates as a browser extension or virtual camera driver and analyzes incoming video streams frame-by-frame, flagging manipulated faces with visual and audio alerts.

Constraints:

- Must process at minimum 15 FPS on a standard CPU (GPU optional)
- False positive rate must not exceed 10% on a clean video benchmark
- Must work with at least one major video call platform (Google Meet, Zoom, or Microsoft Teams via virtual camera) or any free platform you can find
- The model must be lightweight

Expected Output:
A browser extension or system-level virtual camera interceptor with a visible confidence overlay indicating the likelihood of deepfake manipulation in real time.

Judging Emphasis: Speed, accuracy, usability, and integration quality

Track 2 — Social Impact & Inclusion

PS-03 | VaaniDoc: Multilingual AI Health Intake System for Rural Clinics

Difficulty: Medium | Domain: AI + Healthcare + NLP

Background:
Rural and semi-urban clinics in India struggle with patient documentation. Doctors spend

30–40% of consultation time on paperwork. Patients often cannot communicate symptoms effectively due to language and literacy barriers.

Problem:

Build an AI-powered health intake system where patients narrate their symptoms in any Indian regional language (voice or text), and the system auto-generates a structured clinical intake form in English for the doctor — including possible symptom categories and urgency classification.

Constraints:

- Must support majority of Indian languages
- Must work with < 100 KB/s bandwidth (low connectivity rural scenario)
- Must NOT store patient data after session ends (privacy-first design)
- Accuracy on symptom extraction must be validated against 20 test cases

Expected Output:

A mobile-first web application where patients speak or type in their language and receive a generated clinical form, which is simultaneously available to the attending doctor on a shared dashboard.

Judging Emphasis: Language coverage, accuracy of symptom extraction, offline-first capability

PS-04 | LegalAId: AI Legal Rights Assistant for First-Generation Litigants

Difficulty: Medium | Domain: AI + NLP + Civic Tech

Background:

A vast majority of India's population lacks access to legal counsel for everyday legal matters — labor disputes, tenant rights, domestic issues, consumer complaints. Legal documents and rights are written in complex language inaccessible to most citizens.

Problem:

Develop an AI assistant that allows a user to describe their legal situation in plain language (in Hindi or English), and receive: (1) a plain-language explanation of their rights under Indian law, (2) identification of the applicable legal sections, and (3) a templated legal notice or complaint draft.

Constraints:

- Must cover at least 3 domains: consumer rights, labor law, and tenant/rental disputes (the more the merrier)
- Output legal notice must be editable and downloadable as a PDF
- Must cite actual sections of applicable Indian law (IPC/BNS, CrPC, Consumer Protection Act, etc.)
- Must include a "this is not legal advice" disclaimer
- Must be fundamentally different from a GPT wrapper or any existing solutions

Expected Output:
A conversational web application with structured output: rights explanation → applicable law sections → generated document draft.

Judging Emphasis: Legal accuracy, coverage breadth, usability for non-technical users, document quality

Track 3 — Sustainability & Smart Infrastructure

PS-05 | Enterprise Intelligence Platform: Unified Analytics & AI Assistant

Difficulty: High | Domain: AI + Data Engineering + Full Stack

Background:

A private technology company wants a single intelligent platform that demonstrates three core business capabilities: reliable quantitative strategy backtesting, scalable analytical data exploration, and an AI-powered retail assistant. Participants must design and build one integrated platform instead of solving isolated tasks.

Problem:

Build a web platform that includes:

• A Backtesting Module that evaluates trading strategies while preventing look-ahead bias and presenting trustworthy performance metrics.

• A DataMart Analytics Module that ingests transactional datasets, supports fast filtering, aggregation, KPI dashboards, and business insights.

• A Retail AI Assistant that answers customer queries, recommends products, assists shopping workflows, and uses structured business data to generate contextual responses.

Constraints:

• Build all three modules within one unified application.

• Teams are free to choose the technology stack.

• The platform should demonstrate modular architecture, good UI/UX, and scalability.

Expected Output:

A working end-to-end platform with authentication, dashboards, analytics, AI assistant, and a demonstration showing how all three modules work together.

Judging Emphasis:

System architecture, correctness, AI integration, usability, scalability, and code quality.

Track 4 — Developer Tools & Education

PS-06 | CodeOracle: AI-Powered Legacy Codebase Explainer & Modernizer

Difficulty: Medium | *Domain: AI + Software Engineering + Developer Tools*

Background:
Enterprises globally carry millions of lines of legacy code (COBOL, old Java, Python 2) that no current employee fully understands. Knowledge transfer is a major bottleneck in modernization efforts.

Problem:
Build an AI tool that accepts a legacy codebase (uploaded as a ZIP or connected GitHub repo), automatically generates: (1) a natural language explanation of what the code does at module and function level, (2) a dependency graph, (3) auto-generated unit tests, and (4) a proposed modern refactored version with breaking change warnings.

Constraints:

- Must support at least 2 languages: Python (mandatory) + one of Java / JavaScript / C++
- Unit test generation must have > 60% line coverage on provided benchmark scripts
- Explanation quality will be rated by judges on a 1–5 rubric (clarity, accuracy, completeness)
- Must handle codebases up to 10,000 lines without timeout

Expected Output:
A web application with a file upload interface, processing pipeline, and multi-tab output: Explanation | Dependency Graph | Generated Tests | Refactored Code.

Judging Emphasis: Explanation quality, test coverage, refactor safety, scalability

PS-07 | ExamLens: AI Proctoring System with Behavioral Fairness Auditing

Difficulty: Medium-High | *Domain: AI + Education Tech + Ethics*

Background:
Online proctoring tools have proliferated post-pandemic, but most rely on black-box AI flags that disproportionately penalize students with disabilities, poor lighting, or non-standard behaviors. There is no transparency or recourse mechanism.

Problem:
Build an AI-based online exam proctoring system that monitors for genuine cheating signals (screen switching, multiple faces, phone detection), but also includes a built-in fairness audit module that flags potentially biased detections, provides explainability for each flag, and allows examinees to submit a contextual appeal.

Constraints:

- Must run locally (no student data sent to external servers)
- Bias audit must test model outputs across at least 3 demographic proxies (lighting conditions, face angle, presence of glasses/headgear)
- Every automated flag must include a visual explanation (bounding box + confidence score + rule triggered)

- Students must be able to submit a 30-second video appeal attached to a specific flag

Expected Output:
A browser-based exam interface with real-time proctoring, a per-session flag report with explainability visualizations, a fairness audit summary, and an appeal submission workflow for the examiner.

Judging Emphasis: Detection accuracy, fairness audit depth, explainability quality, appeal mechanism design

Hackathon Rules & Guidelines

1. **Team Composition:** Teams must have 3–5 members. Solo participation is not allowed. Inter-department teams are strongly encouraged.
2. **Problem Exclusivity:** Each team may work on only one problem statement. First-come-first-served registration locks the problem choice.
3. **Technology Stack:** Participants are free to use any programming language, framework, or cloud platform. Use of proprietary APIs (OpenAI, Gemini, etc.) is allowed but must be declared in the final submission.
4. **Open Source:** All third-party libraries must be open source or have free-tier licenses for academic use.
5. **Originality:** All code must be written during the hackathon. Pre-existing personal projects cannot be submitted. Use of boilerplate templates or starter kits (official or shared by organizers) is permitted.
6. **Submission Requirements:**
    - Public GitHub repository with a descriptive README
    - 3-minute demo video (uploaded to YouTube/Drive)
    - 1-page project abstract (submitted via Google Form)
    - Working **AND** deployed prototype
7. **Conduct:** Respectful collaboration is mandatory. Plagiarism or misrepresentation of work results in immediate disqualification.
8. **Mentorship:** Each team may access mentors for up to 30 minutes total during the hackathon. Mentor inputs should be advisory only.